

FCN과 Image Segmentation

학생 수업용 단계별 학습자료

Classification → Segmentation → FCN → Upsampling → Decoder → 1×1 Convolution

학습 목표

- ① 이미지 분류와 픽셀 단위 분할의 차이를 이해한다.
- ② FCN이 픽셀별 예측을 할 수 있는 이유를 설명한다.
- ③ Upsampling과 Decoder의 역할을 이해한다.
- ④ 1×1 Conv가 feature를 클래스 점수로 바꾸는 원리를 이해한다.

핵심: Segmentation은 이미지 전체에 하나의 답을 주는 것이 아니라 이미지의 각 위치에 답을 주는 문제이다.

1. Classification부터 이해하기

이미지 Classification은 이미지 전체를 보고 하나의 클래스를 예측한다.

고양이 사진 → cat

개 사진 → dog

즉, 질문은 “이 이미지에는 무엇이 있는가?”이다.

2. Segmentation은 무엇이 다른가?

Semantic Segmentation은 각 픽셀마다 어떤 클래스에 속하는지 예측한다. 224×224 이미지라면 50,176개의 위치 각각에 클래스가 부여된다.

Classification : 이미지 전체 → cat

Semantic Segmentation : 각 픽셀 → background / cat / dog

따라서 3개 클래스를 분류한다면 최종 출력은 예를 들어 224×224×3이다. 여기서 3은 RGB가 아니라 클래스 수이다.

3. CNN에서 왜 공간 정보가 줄어드는가?

Pooling이나 stride를 사용하면 feature map의 공간 크기가 줄어든다.

224×224 → 112×112 → 56×56 → 28×28

공간 크기를 줄이면 계산량이 감소하고 넓은 영역의 의미를 파악하기 쉬워지지만, 세밀한 위치 정보는 감소한다.

Segmentation에서는 이 위치 정보가 중요하다.

4. FCN(Fully Convolutional Network)의 핵심

FCN은 공간 정보를 유지하는 convolution 중심 구조를 사용하여 픽셀 단위 예측을 가능하게 한다.

입력 이미지 → Convolution/Pooling → 작은 feature map → Upsampling/Decoder → 큰 feature map → 1×1 Conv
→ 픽셀별 클래스 점수

중요: 픽셀별 예측이라고 해서 해당 픽셀의 RGB만 보는 것은 아니다. 각 위치의 feature에는 주변 영역과 더 넓은 영역에서 얻은 정보가 포함된다.

5. Upsampling

Pooling/stride로 작아진 feature map의 공간 해상도를 다시 키우는 과정이다.

$28 \times 28 \rightarrow 56 \times 56 \rightarrow 112 \times 112 \rightarrow 224 \times 224$

Nearest/Bilinear 같은 보간법이나 학습 가능한 Transposed Convolution 등을 사용할 수 있다.

주의: Upsampling이 사라진 원본 정보를 그대로 복구하는 것은 아니다. Decoder convolution이나 U-Net의 skip connection 등이 세부 정보를 보완한다.

6. Decoder의 원리

Encoder는 특징을 추출하고 공간 크기를 줄인다.

Decoder는 feature를 이용해 공간 해상도를 키우고 픽셀 단위 예측이 가능하도록 feature를 정교화한다.

U-Net의 Skip Connection은 Encoder의 얇은 층 정보를 Decoder에 직접 전달한다. 얇은 층은 경계·세부 위치 정보, 깊은 층은 고수준 의미 정보를 상대적으로 잘 보존한다.

7. Decoder가 224×224 가 되면 어떻게 각 픽셀을 분류하는가?

Decoder의 출력이 $224 \times 224 \times 64$ 라고 하자. 이것은 224×224 개의 공간 위치가 있고, 각 위치마다 64개의 feature 값이 있다는 뜻이다.

$(100, 100)$ 위치 $\rightarrow x = [x_1, x_2, \dots, x_{64}]$

즉, Decoder는 원본 RGB 이미지를 다시 만드는 것이 아니라 각 위치에 대한 64차원 feature vector를 만든다고 이해하는 것이 정확하다.

8. 1×1 Convolution이 하는 일

입력: $224 \times 224 \times 64$

1×1 Conv

출력: $224 \times 224 \times 3$

1×1 Conv는 한 공간 위치에서 64개의 feature를 받아 하나의 점수를 만들 수 있다. 클래스가 3개라면 이런 필터를 3개 사용한다.

Filter 1 $\rightarrow$ background 점수

Filter 2 $\rightarrow$ cat 점수

Filter 3 $\rightarrow$ dog 점수

224×224 는 그대로이고 $64 \rightarrow 3$ 으로 채널 수만 바뀐다. 마지막 3은 RGB가 아니라 segmentation 클래스 수이다.

9. 1×1 Conv를 수학적으로 이해하기

한 위치의 feature를 $x \in \mathbb{R}^64$ 라고 하면 각 클래스 점수는 다음과 같이 계산된다.

$z_1 = w_1 \cdot x + b_1 \rightarrow \text{background}$

$z_2 = w_2 \cdot x + b_2 \rightarrow \text{cat}$

$z_3 = w_3 \cdot x + b_3 \rightarrow \text{dog}$

따라서 1×1 Conv는 직관적으로 각 픽셀 위치에서 64차원 feature를 3개의 클래스 점수로 바꾸는 작은 분류기이다. 같은 가중치를 모든 위치에 적용한다(weight sharing).

10. 실제 픽셀 하나에서 일어나는 일

$(100, 100)$ 위치의 결과가 다음과 같다고 하자.

background = 0.2

cat = 3.8

dog = 0.5

가장 큰 값이 cat이므로 (100,100)은 cat으로 분류된다. 이 계산을 224×224의 모든 위치에서 수행하여 224×224 segmentation mask를 만든다.

11. Softmax를 적용하면 확률이 된다

각 위치의 3개 점수(logit)에 Softmax를 적용하면 클래스별 확률이 된다.

(100,100) → [background 0.03, cat 0.92, dog 0.05] → 최종 클래스 = cat

전체적으로 224×224×3은 각 위치의 3개 클래스 점수를 나타내며, argmax를 취하면 최종 224×224 클래스 맵을 얻는다.

12. 학습은 어떻게 이루어지는가?

정답 segmentation mask가 필요하다. 예: 0=background, 1=cat, 2=dog.

모델 예측과 정답을 비교하여 픽셀별 Cross-Entropy Loss를 계산하고, Backpropagation으로 1×1 Conv뿐 아니라 Decoder와 Encoder의 가중치까지 업데이트한다.

즉, 모델은 반복 학습하면서 어떤 feature 조합이 어떤 클래스를 나타내는지 스스로 학습한다.

13. 전체 구조를 한 번에 보기

① 224×224×3 입력
↓
② Encoder: Convolution + Pooling
↓
③ 예: 28×28×512
↓
④ Decoder/Upsampling
↓
⑤ 예: 224×224×64
↓
⑥ 1×1 Conv
↓
⑦ 224×224×3 클래스 점수
↓
⑧ Softmax
↓
⑨ 224×224 픽셀별 segmentation mask

14. 학생들이 반드시 구분할 것

- 224×224×64의 64 = 각 위치의 feature 개수
- 마지막 3 = segmentation 클래스 수
- 1×1 Conv = 각 위치의 feature vector → 클래스 점수
- Upsampling = 공간 해상도 증가
- Decoder = 해상도 증가 + 픽셀 예측에 필요한 feature 정교화
- FCN = 공간 정보를 유지하는 convolution 중심 구조로 픽셀 단위 예측

15. 수업용 비유

Encoder = 압축하면서 “무엇인지” 알아내기

Upsampling = 작아진 공간 크기를 다시 키우기

Decoder = 위치와 형태를 다시 정교하게 만들기

1×1 Conv = 각 위치에서 “이 feature는 어떤 클래스인가?” 판정하기

전체 흐름: 이미지 → feature 추출 → 작은 feature map → Upsampling/Decoder → 각 위치의 feature → 1×1 Conv
→ 클래스별 점수 → 픽셀별 segmentation

16. 수업용 확인 문제

- 1) Classification과 Semantic Segmentation의 출력 차이는?
- 2) Pooling을 반복하면 공간 크기가 왜 작아지는가?
- 3) Upsampling이 필요한 이유는?
- 4) Decoder가 원본 RGB 이미지를 그대로 복원하는가?
- 5) 224×224×64에서 5개 클래스를 분류한다면 1×1 Conv 출력 shape은?
- 6) 224×224×64 → 224×224×3에서 마지막 3의 의미는?
- 7) 1×1 Conv가 각 위치에서 수행하는 계산은?

17. 최종 요약

Segmentation의 핵심은 “이미지의 어느 위치에 무엇이 있는가?”를 알아내는 것이다. Encoder는 feature를 추출하면서 공간 크기를 줄이고, Decoder/Upsampling은 공간 해상도를 키운다. Decoder의 224×224×64 출력에서는 각 위치에 64개의 feature가 있다. 1×1 Conv는 이를 클래스별 점수로 바꾼다. 클래스가 3개라면 224×224×3이 되고, 각 위치에서 Softmax를 적용하면 3개 클래스의 확률을 얻는다.